

Route Statistics — Feature Specification

Status: Backend Complete — Portal UI Pending **Date:** 2026-04-20 **Scope:** Backend API + Portal dashboard section

Implementation Status

Layer	Status
DB query files (atomic)	20/20 implemented — session + date-range for all 10 metrics
Handlers (one per query file, in api.ts)	20/20 implemented
Route registrations (20 routes in app.ts)	20/20 implemented
Portal fetch service (routeStatsFetchService.ts)	Pending — backend ready to consume
Portal UI component (RouteStatsSection.tsx)	Pending
Portal page embed (page.tsx)	Pending

Metric heading key: Implemented · Pending · Deferred

All backend work is complete and live. The 20 endpoints are registered and functional. Portal UI integration is the remaining work.

DB Query Files (as of 2026-04-03)

Each metric has two files — one for session scope, one for date-range scope. All files are in `backend/src/services/db/DatabaseOperationsServices/Read/`.

Metric	Session file	Date-range file
4.1 Total Routes Completed	getTotalRoutesCompleted.ts	getTotalRoutesCompletedByDateRange.ts
4.2 Total Connected Users	getTotalConnectedUsers.ts	getTotalConnectedUsersByDateRange.ts
4.3 Users Started >1 Route	getUsersStartedMoreThanOneRoute.ts	getUsersStartedMoreThanOneRouteByDateRange.ts
4.4 Users Completed >1 Route	getUsersCompletedMoreThanOneRoute.ts	getUsersCompletedMoreThanOneRouteByDateRange.ts
4.5 Users Never	getUsersNeverStartedRoute.ts	getUsersNeverStartedRouteByDateRange.ts

Started		
4.6 Started But Not Finished	getuserswhostartedbutdidnotfinishanyroute.ts	getUsersStartedButNotFinishedByDateRange
4.7 Total Replays	getTotalReplays.ts	getTotalReplaysByDateRange.ts
4.8 Avg Routes / User	getAvgRoutesPerUserBySession.ts	getAvgRoutesPerUserByDateRange.ts
4.9 Avg Routes / Session	getAvgRoutesPerSessionBySession.ts	getAvgRoutesPerSessionByDateRange.ts
4.10 Replay Ratio	getReplayRatioBySession.ts	getReplayRatioByDateRange.ts

1. Purpose

The portal currently only surfaces survey demographics (party size data). This feature adds a second view — **Route Statistics** — so staff can understand how players actually engage with the game: how many routes they complete, who replays, who drops out, and whether accessibility features are being used.

2. Data Sources

All metrics except accessibility derive from three existing tables:

Table	Key fields used
Session	id, sessionDate
PlayerSession	id, sessionId, playerId
RouteCycle	sessionId, playerSessionId, cycleNumber, wasCompleted, startedAt, completedAt, completionMs

Accessibility requires a **new field** on `PlayerSession` (see §8).

3. Scope Modes

All metrics support two scope modes, matching the existing portal search pattern:

Mode	Input	Description
Session	sessionId: number	Metrics scoped to one session
Date range	startDate, endDate (optional)	Metrics aggregated across all sessions in window

4. Metric Definitions

4.1 Total Routes Completed

Definition: Number of `RouteCycle` records where `wasCompleted = true` .

Source:

```
SELECT COUNT(*) FROM RouteCycle
WHERE sessionId = ? AND wasCompleted = true
```

Why not use `Session.totalRoutesCompleted` ? That column is a cached/legacy field that is not reliably updated. Always compute live from `RouteCycle` .

4.2 Total Connected Users

Definition: Number of distinct players who connected to the session — i.e., the count of `PlayerSession` rows for the session.

Each `PlayerSession` row is created the first time a player calls `/api/start-game` , so this count reflects how many devices/players actually started the app during the session.

Source:

```
SELECT COUNT(*) FROM PlayerSession
WHERE sessionId = ?
```

4.3 Users Who Started More Than 1 Route

Definition: Number of `PlayerSession` records linked to **more than one** `RouteCycle` row for the session.

A new `RouteCycle` row is created each time a player is assigned a route — whether it is a fresh route or a replay. This metric counts players who received and began at least two route assignments in a session, regardless of whether they completed any.

Relationship to other metrics:

- Differs from §4.4: §4.4 requires both (or more) routes to be *completed*. A player who started two routes but finished neither is counted here but not in §4.4.
- Differs from §4.7: §4.7 counts replays specifically (`cycleNumber > 1`). A player who was assigned two *different* new routes (no replay) is counted here but not in §4.7. A player who triggered a replay is counted in both.

Source:

```
SELECT COUNT(*) FROM (
  SELECT playerId
  FROM RouteCycle
  WHERE sessionId = ?
  GROUP BY playerId
  HAVING COUNT(*) > 1
) sub
```

API field: `usersStartedMoreThanOneRoute`

4.4 Users Who Completed More Than 1 Route

Definition: Number of `PlayerSession` records that have **more than one** `RouteCycle` with `wasCompleted = true`.

These are players who fully completed at least two distinct routes within one session.

Source:

```
SELECT COUNT(*) FROM (
  SELECT playerSessionId
  FROM RouteCycle
  WHERE sessionId = ? AND wasCompleted = true
  GROUP BY playerSessionId
  HAVING COUNT(*) > 1
) sub
```

API field: `usersCompletedMoreThanOneRoute`

4.5 Users Who Never Started Any Route

Definition: Number of `PlayerSession` records that have **no** `RouteCycle` rows at all.

These are players who connected to the session (called `/api/start-game`) but never began navigating a route.

Source:

```
SELECT COUNT(*) FROM PlayerSession ps
WHERE ps.sessionId = ?
AND NOT EXISTS (
  SELECT 1 FROM RouteCycle rc
  WHERE rc.playerSessionId = ps.id
)
```

API field: `usersNeverStartedRoute`

4.6 Users Who Started But Did Not Finish Any Route

Definition: Number of `PlayerSession` records that have **at least one** `RouteCycle` row but **zero** rows where `wasCompleted = true`.

These are players who began navigating at least one route but left before completing it.

Source:

```
SELECT COUNT(*) FROM PlayerSession ps
WHERE ps.sessionId = ?
AND EXISTS (
  SELECT 1 FROM RouteCycle rc
  WHERE rc.playerSessionId = ps.id
)
```

```

)
AND NOT EXISTS (
  SELECT 1 FROM RouteCycle rc
  WHERE rc.playerSessionId = ps.id AND rc.wasCompleted = true
)

```

API field: `usersStartedButNotFinished`

4.7 Total Replays

Definition: Number of `RouteCycle` records where `cycleNumber > 1`.

A new cycle (`cycleNumber` increments) is started when a player has seen every available route at least once and is assigned a route they have already completed. Each such assignment is a replay.

Source:

```

SELECT COUNT(*) FROM RouteCycle
WHERE sessionId = ? AND cycleNumber > 1

```

Related: "Users who triggered a replay" = `COUNT(DISTINCT playerSessionId WHERE cycleNumber > 1)`. Both values are returned by the `/api/route-stats/total-replays` endpoint.

4.8 Average Route Completions per User

Definition:

$$\frac{\text{total completed routes (wasCompleted = true)}}{\text{total connected users (PlayerSession count)}}$$

Returns `0` if there are no connected users.

Returns a float rounded to 2 decimal places.

4.9 Average Route Completions per Session

Definition:

$$\frac{\text{total completed routes across all sessions in scope}}{\text{number of distinct sessions in scope}}$$

Only meaningful in date-range mode (always equals 4.1's value in single-session mode). Still returned in both modes for completeness.

Returns `0` if there are no sessions. Returns a float rounded to 2 decimal places.

4.10 Replay Ratio

Definition (player-based):

```
players who triggered at least one replay (cycleNumber > 1)
```

```
total connected users
```

Definition (cycle-based):

```
RouteCycle records with cycleNumber > 1
```

```
total RouteCycle records
```

Both ratios are returned by the `/api/route-stats/replay-ratio` endpoint. Expressed as a float `[0, 1]` rounded to 4 decimal places.

The **player-based ratio** answers: "What fraction of players ran out of new routes?" The **cycle-based ratio** answers: "What fraction of all route plays were replays?"

5. API Contract

Design: One Endpoint per Query

Each metric has two endpoints — one for session scope, one for date-range scope. 20 endpoints total. The portal calls whichever scope endpoint matches the user's search input. Every request hits the database directly; no caching layer is used.

All endpoints:

- Method: `POST`
- Auth: None (matches existing portal pattern)
- Return the same envelope shape, with `metric` and `value` fields specific to each endpoint

Request Bodies

Session endpoints — accept:

```
{ "sessionId": 202401010 }
```

Date-range endpoints — accept:

```
{ "startDate": "2026-01-01", "endDate": "2026-03-30" }
```

`endDate` is optional — if omitted, defaults to no upper bound.

Common Response Envelope (200)

```
{  
  "success": true,  
  "scope": "session",  
  "scopeValue": 202401010,  
  "metric": "<metric-name>",  
}
```

```
"value": <number or object>
}
```

`scope` is `"session"` or `"dateRange"` . `scopeValue` mirrors the input.

Endpoints

Session endpoint	Date-range endpoint	Metric	value type
POST /api/route-stats/total-routes-completed-by-session	POST /api/route-stats/total-routes-completed-by-date-range	4.1	number
POST /api/route-stats/total-connected-users-by-session	POST /api/route-stats/total-connected-users-by-date-range	4.2	number
POST /api/route-stats/users-started-more-than-one-route-by-session	POST /api/route-stats/users-started-more-than-one-route-by-date-range	4.3	number
POST /api/route-stats/users-completed-more-than-one-route-by-session	POST /api/route-stats/users-completed-more-than-one-route-by-date-range	4.4	number
POST /api/route-stats/users-never-started-route-by-session	POST /api/route-stats/users-never-started-route-by-date-range	4.5	number
POST /api/route-stats/users-started-but-not-finished-by-session	POST /api/route-stats/users-started-but-not-finished-by-date-range	4.6	number
POST /api/route-stats/total-replays-by-session	POST /api/route-stats/total-replays-by-date-range	4.7	{ totalReplays: number, replayUsersCount: number }
POST /api/route-stats/avg-routes-per-user-by-session	POST /api/route-stats/avg-routes-per-user-by-date-range	4.8	number
POST /api/route-stats/avg-routes-per-session-by-session	POST /api/route-stats/avg-routes-per-session-by-date-range	4.9	number
POST /api/route-stats/replay-ratio-by-session	POST /api/route-stats/replay-ratio-by-date-range	4.10	{ byPlayer: number, byRoute: number }

Example Responses

POST /api/route-stats/total-routes-completed-by-session

```
{
  "success": true,
  "scope": "session",
  "scopeValue": 202401010,
  "metric": "totalRoutesCompleted",
  "value": 47
}
```

POST /api/route-stats/total-replays-by-date-range

```
{
  "success": true,
  "scope": "dateRange",
  "scopeValue": { "startDate": "2026-01-01", "endDate": "2026-03-30" },
  "metric": "totalReplays",
  "value": { "totalReplays": 3, "replayUsersCount": 2 }
}
```

POST /api/route-stats/replay-ratio-by-session

```
{
  "success": true,
  "scope": "session",
  "scopeValue": 202401010,
  "metric": "replayRatio",
  "value": { "byPlayer": 0.1667, "byRoute": 0.0638 }
}
```

Error Responses (all endpoints)

Status	Body	When
400	{ "error": "sessionId or startDate is required" }	No valid input provided
400	{ "error": "Invalid sessionId" }	Not a number
400	{ "error": "Invalid date format – use YYYY-MM-DD" }	Date not YYYY-MM-DD
500	{ "error": "Failed to compute statistics", "message": "..." }	DB error

6. Portal UI

Location

A second tab or collapsible section on the existing portal dashboard ([portal/src/app/page.tsx](#)), labelled "Route Statistics".

Search Input

Reuses the same Session ID / Date Range search pattern already in the portal. No new input validation needed — same `inputValidator.ts` functions apply.

Data Loading

The portal component calls each metric endpoint independently. This means:

- Metrics can render as they arrive (no need to wait for all queries)
- A slow query for one metric does not block display of others
- Each metric row shows a loading indicator until its fetch resolves

Output Layout

ROUTE STATISTICS		
Session 202401010 – 2026-03-01		
Total Routes Completed		47
Total Connected Users		12
Users Started > 1 Route		9
Users Completed > 1 Route		8
Users Never Started a Route		0
Users Started, Didn't Finish		1
Total Replays		3
Users Who Replayed		2
Avg Routes / User		3.92
Avg Routes / Session		47.00
Replay Ratio (Players)		16.67%
Replay Ratio (Routes)		6.38%

Ratios display as percentages in the UI (multiply float by 100, format to 2dp).

7. Out of Scope

- Per-route breakdown (which routes are most/least completed) — future feature
- Completion time distribution — `completionMs` is captured in `RouteCycle` but not surfaced here
- Real-time / live updates — portal is pull-based (search on demand); data is always ≥24 hours old

8. Open Questions

#	Question	Default assumption
1	Should "connected users" count players who only validated (hit <code>/api</code>) but never called <code>/api/start-game</code> ?	No — <code>PlayerSession</code> is only created on <code>start-game</code> , so validation-only players are not counted

2	Should replays count individual route cycles or unique player-route pairs?	Cycles (RouteCycle rows with cycleNumber > 1)
3	For the replay ratio, which definition (player-based or cycle-based) should be the headline number?	Return both; portal shows player-based as primary
4	Date-range scope: should dates filter on Session.sessionDate or RouteCycle.createdAt?	Session.sessionDate — consistent with the existing session model